Technical Documentation

Depression Indicator Prediction Using NHANES Data

MACHINE LEARNING PIPELINE AND CODE DOCUMENTATION

Repository: melancholy-modeling (https://github.com/nchaudhry0808/melancholy-modeling)
Group: Group 3

Team: Angela Johnson, Annabel Tapia, Naeem Chaudhry, Thant Thiri Myo Kyi
Date: August 2026

Table of Contents:

1. Introduction

1.1 Purpose

The purpose of this technical document is to describe the structure, implementation and machine learning pipeline of the Depression Indicator Prediction project. It provides information about the project files, data preparation, feature engineering, model development, evaluation, and dependencies so that the code and overall pipeline can be understood, tested, reproduced, and maintained.

The project was completed collaboratively by Group 3: Angela Johnson, Annabel Tapia, Naeem Chaudhry, and Thant Thiri Myo Kyi. Team members contributed to different parts of the project, including data preparation, exploratory data analysis, feature engineering, machine learning model development, model comparison, documentation, and the final presentation.

This project uses machine learning to predict whether a survey respondent is likely to screen positive for depression simply based on demographic, socioeconomic, and lifestyle information from the National Health and Nutrition Examination Survey (NHANES). The purpose of this project is for education and research use. The model is meant to be used as a screening tool and not as a medical diagnosis or replacement for evaluation by a healthcare professional.

1.2 Definitions, Acronyms, and Abbreviations

Term	**Definition**
NHANES	National Health and Nutrition Examination Survey
PHQ-9	Patient Health Questionnaire-9, a nine-question depression screening questionnaire
EDA	Exploratory Data Analysis
SVM	Support Vector Machine
XGBoost	Extreme Gradient Boosting
CatBoost	A gradient boosting machine learning algorithm that can directly handle categorical features
PR-AUC	Area Under the Precision-Recall Curve
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
F1 Score	A metric that combines precision and recall into a single score
CSV	Comma-Separated Values file
XPT	SAS transport file format used for the raw NHANES data

Term	Definition
Depression_Flag	Binary target variable indicating whether a respondent screened positive for depression

1.3 Machine Learning Problem

This project is a supervised binary classification problem. Each respondent is classified as either likely to screen positive for depression or not likely to screen positive for depression. We trained and compared five machine learning models: Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, and CatBoost. Each model uses the respondent's survey information to predict the depression screen outcome.

1.4 Target Variable

The target variable for this project is **Depression_Flag**. It is a binary variable created using responses from the NHANES Patient Health Questionnaire-9 (PHQ-9), which includes nine questions related to depression symptoms (DPQ010 through DPQ090).

Each question is scored from 0 to 3, and the responses are added together in order to create PHQ9_Score, which can range from 0 to 27. A respondent is assigned to a Depression Flag of 1 if their PHQ9_Score is 10 or higher and 0 if their score is below 10. Respondents who did not answer all nine PHQ-9 questions are excluded instead of estimating or filling in their missing score.

Since the target is based on a screening questionnaire and not an official medical diagnosis, the model predicts whether someone screens positive for depression, rather than saying that the person has depression.

1.5 NHANES Data Used

For this project, we used data from two NHANES cycles. The raw data was collected from SAS transport (.xpt) files:

- 2017-March 2020 (Pre-Pandemic) : files use the _P suffix and represent approximately 3.2 years of data collection
- August 2021-August 2023 : files use the _L suffix and represent approximately 2 years of data collection

For each cycle, data was collected from eight questionnaire topics:

NHANES File Prefix	Topic
DEMO	Demographics, including age, gender, race/ethnicity, education, marital status, country of birth, income-to-poverty ratio, and survey weights

DPQ	Depression screening (PHQ-9) and the source used to create the target variable
ALQ	Alcohol use
SMQ	Smoking
SLQ	Sleep disorders and sleep duration
PAQ	Physical activity
HSQ	General health status
INQ	Income

Before any cleaning or filtering, the 2017-March 2020 cycle contains 15,560 respondents, while the 2021-2023 cycle contains 11,933 respondents, giving us a combined total of 27,493 respondents.

Our original project plan focused only on the 2021-2023 NHANES cycle. During the project, we expanded the dataset to include both cycles. Combining the two cycles gave us a larger sample size to work with, especially after removing respondents who did not have a complete PHQ-9 response.

1.6 Relationship to the Original Project Plan

Our original Project Topic Form proposed using NHANES 2021-2023 data and factors such as age, sleep duration, physical activity, BMI, education, employment, income, general health, chronic disease, and healthcare access. The original plan included using Random Forest, XGBoost, Support Vector Machine (SVM), and CatBoost as our machine learning models.

As we worked through this, we made a few changes to the original plan. We expanded the dataset from one NHANES cycle to two pooled cycles to increase the amount of data available for modeling. We also added Logistic Regression as a baseline model so we could compare its performance with the other four models.

Some of the predictors included in our original plan were also not included in the final modeling dataset. These include BMI, employment, general health, chronic diseases, and healthcare access. These differences are documented in *Models/project_goal_check.csv* and are included as limitations of the final project.

2. Technical Documentation

The repository is organized around a series of numbered Jupyter notebooks that follow the main steps of our machine learning pipeline. Supporting folders are also used to store the raw data, data splits, trained models, figures, and project documentation.

2.1 Important Files

The main notebooks are organized in the order that they are used throughout the project;

Order	**Notebook**	**Programmer/Contributor**	**Pipeline Stage**
1	*1.RawDataMapping.ipynb*	Thant Thiri Myo Kyi	Loads the raw NHANES.xpt files, maps the selected variables, merges the topic files for each cycle, combines the two cycles, and calculates the pooled survey weights
2	*2.DataWrangling.ipynb*	Annabel Tapia	Cleans missing and invalid values, creates the PHQ-9 score and Depression_Flag target variable, removes unusable columns, handles remaining missing values, and saves the cleaned dataset
3	*3.EDA.ipynb*	Angela Johnson	Explores the cleaned dataset, including its structure, distributions, class balance, and correlations, to better understand the data before modeling
4	*4.FeatureEngineering.ipynb*	Naeem Chaudhry	Prepares the data for modeling by removing the zero-variance feature, creating the train, validation, and test splits, and creating the different feature encodings needed by the models
5	*5.Modeling_rf_logreg.ipynb*	Thant Thiri Myo Kyi	Trains and tunes the Logistic Regression and Random Forest models
5	*5.Modeling_svm.ipynb*	Naeem Chaudhry	Trains and tunes the

			Support Vector Machine (SVM) model
5	*5.Modeling_XGBoost.ipynb*	Annabel Tapia	Trains and tunes the XGBoost model
5	*5.Modeling_CatBoost.ipynb*	Angela Johnson	Trains and tunes the CatBoost model
6	*6.ModelComparison.ipynb*	Thant Thiri Myo Kyi	Compares the performance of all five models, reviews the final results, and selects the final model

2.2 Supporting Folders and Files

In addition to the main notebooks, the repository contains supporting folders and files used throughout the project:

Path	**Contents**
Nhanes_RawData/	Contains the raw NHANES.xpt files and supporting codebook PDFs. It also contains the merged, mapped, and cleaned datasets created during the raw data mapping and data wrangling stages.
DataSplit/	Contains the train, validation, and test datasets created during feature engineering, including the different versions needed for CatBoost, tree-based models, and standardized models.
Models/	Contains the five trained model files, the selected final model, model card, scoreboards, feature importance results, coefficients, comparison results, and other model evaluation outputs.
Figures/	Contains the saved visualizations produced during modeling and model comparison, including feature importance plots, coefficient plots, ROC and precision-recall curves, and the confusion matrix.
Archives/	Contains an earlier version of the raw data mapping notebook. This notebook is kept for reference but is not part of the final pipeline.

docs/	Contains project planning documents, including the Project Topic Form and Project Progress Report(s). It also contains preview.html, which is a prototype of a possible depression assessment interface and is separate from the machine learning pipeline.
Untitled.ipynb	Empty notebook, not used as part of the project pipeline

3. Raw Data Mapping - 1.RawDataMapping.ipynb

This notebook is the entry point of the pipeline. It converts the raw NHANES files into two artifacts used downstream: a fast numeric CSV for the cleaning notebook, and a human-readable Excel workbook for reference.

3.1 Data Loading

The notebook loads 16 raw files with 8 survey topics (DEMO, DPQ, ALQ, SMQ, SLQ, PAQ, HSQ, INQ), each collected in both the 2017-March 2020 cycle and the 2021-2023 cycle. Each file is read in, trying a standard text format first and falling back to the backup format if needed. The notebook figures out which topic and which cycle a file belongs to by just using its file name; for example, DPQ_L.xpt is the Depression file from the 2021-2023 cycle. The two cycles are not the same size; for example, the Demographics file has 15,560 people in the older cycle but only 11,933 in the newer one because the older cycle covered a longer collection period.

3.2 Value Mapping (Construction of Code)

The raw files already include the question text for each variable but not what each answer code means that had to be typed by hand from the NHANES official documentation. In total, 74 variables have their answer codes mapped this way. For example, code 1 means “Yes” and code 7 means “Refused”. Separately, 24 number-based variables needed their own special handling because a few of them use large placeholder numbers to mean “Refused” or “Don’t know” instead of a real value. For example a sitting time variable uses 9999 to mean “Don’t know” not 9,999 minutes. Recording these placeholder codes here keeps a complete and accurate record of what every number in the data actually means.

3.3 Merging and Stacking

Each person’s ID number only stays unique within one cycle not across both, so the eight topic files are combined together separately for each cycle, first always starting from the Demographics file so no one gets dropped just because they are missing from another topic. Weight columns are also renamed at this point so both cycles use the same column names. This produces one table for 2017-March 2020 (15,560 people, 97 columns) and one for 2021-2023 (11,933 people, 76 columns), which are then stacked into a

single combined table of 27,493 people and 116 columns. Three extra columns keep track of which cycle each person originally came from.

3.4 Pooled Survey Weights

NHANES survey weights are scaled based on how many years of data each cycle covers, about 3.2 years for the older cycle and 2.0 years for the newer one for a combined window of 5.2 years. Each person's weight is adjusted based on that ratio so people from each cycle are represented fairly in the combined dataset. After this step, the combined weights add up to about 324 million across all 27,493 respondents with no one missing a weight.

3.5 Quality Checks

The notebook runs eight automatic checks at the end and all eight checks passed. The total row count after combining matches exactly what was expected: no one's ID number is duplicated within a single cycle, each cycle only has one survey cycle code, the nine depression questions are present in both cycles, and every single person has both survey weights filled in. The checks also confirm that some variables only exist in one cycle and not the other which is what we expected.

3.6 Outputs

By the end, the notebook has built a reference table of 758 rows, one row per variable and answer code. For each variable, it notes whether it is available in both cycles, 52 variables. The next steps are to decide which variables are usable across the full combined dataset. Two files are saved: a plain CSV file with the raw combined data (27,493 people and 118 columns) and an excel workbook with the full reference table plus a readable, labeled sheet for each topic and cycle meant for humans to look at rather than the code to use.

4. Data Cleaning and Preprocessing - 2.DataWrangling.ipynb

This notebook turns the raw merged table from Notebook 1 into the single model-ready table used by every downstream notebook. It works mainly on the numeric code files (Nhanes_Merged_Raw.csv) and mirrors every change onto a parallel decoded/labeled copy to a human-readable version.

4.1 Sentinel Missing-Value Cleanup

The notebook first loads the codebook including the raw numeric data and the readable labeled version then double-checks that both copies line up person-to-person before doing anything else. For every column, it looks up which numbers mean "Refused" or "Don't know" and replaces those with real blanks instead of a number using the codebook as a lookup table. This step found placeholder codes in 80 columns and blanked out 4,558 values total. Every blank in the numbers version is mirrored onto the readable copy too so both stay in sync.

4.2 Target Construction

The 9 depression screening questions are added together into one score from 0-27 but only for people who answered all 9. If someone skipped any of them, their scores are left blank rather than guessed. A score of 10 or higher is marked as a "likely depression" flag and anything below is marked as "not flagged." Before removing anyone, this gave 12,241 people flagged "not depressed", 1,490 people flagged "likely depression", and 13,762 people left blank because they did not answer all 9 questions.

4.3 Row Filtering

Anyone without a usable flag from the step above is dropped since a model cannot be trained on someone whose actual outcome isn't known. This brings the dataset from 27,493 people down to 13,731 people 8,276 from the pre-pandemic data and 5,455 from the newer data. The result is noticeably imbalanced: 89.1% of people are not flagged and 10.9% are. There's also a real difference between the two time periods about 9.3% flagged in pre-pandemic versus about 13.3% in the newer cycle which is worth noting since it is a genuine shift over time and not a data error.

4.4 Column Dropping

The notebook checks how much of each column is missing both overall and separately for each cycle. This matters because pooling two cycles together can make a column look "half missing" simply because it was only ever asked in one of the two cycles not because of any real data quality problem. Based on this, columns get dropped for five reasons: only asked in one cycle, more than half missing, one of the 9 depression questions that were used to build the target itself, survey design/weight columns that describe how NHANES samples people rather than anything about the person, and a small number of duplicate/redundant columns. In total, 92 columns were dropped bringing the table down to 13,731 rows and 28 columns.

4.5 Missing Value Imputation

The columns left after dropping are split into either continuous which is a plain number such as age (8 columns) and categorical which is like gender or education level (15 columns). Continuous columns have their remaining blanks filled with the middle median value. Categorical columns get a new code, -1 meaning "not answered" instead of guessing a real category for them. The same fill is mirrored onto the readable copy where categorical blanks become the text "Not answered". After this step there are zero missing values left anywhere in the table.

4.6 Final Checks and Save

Out of the 28 remaining columns, 23 are marked as usable for model features. The other 5 are kept in the file for reference but are not meant to be fed into a model. These 5 are the respondent ID, the cycle label (used only for splitting/reporting, not as an input), the survey weight (used only for weighted reporting), the raw PHQ-9 score, and the depression flag itself. Eight automated checks ran at the end and all passed confirming the target is clean 0/1 values, there's no missing data left among the features, none of the raw depression questions leaked back in as a feature, every feature exists in both cycles, both cycles are

represented in the final data, and no single feature can predict the outcome on its own. A quick comparison also showed the weighted depression rate (10.06%) and the plain unweighted rate (10.85%) are close to each other which is a good sign.

4.7 Outputs

Two files are saved: Nhanes_Cleaned.csv which is the plain number version that every modeling notebook actually loads and the Nhanes_Cleaned.xlsx which holds two matching row-for-row identical tabs one with readable labels for charts and the written report and one with the same data in raw numeric form. At the time this notebook output was saved, the cleaned dataset contained 13,731 rows and 28 columns: 23 usable features plus 5 reference-only columns. A later update removed three reference columns (SEQN, CYCLE, and PHQ9_Score), leaving 25 columns in the current cleaned dataset.

5. Exploratory Data Analysis - 3.EDA.ipynb

The purpose of the EDA notebook was to better understand the cleaned dataset before moving onto the feature engineering and modeling. We looked at the structure of the data, class balance, feature distributions, relationships with the target variable, correlations, and low-variance features.

Note: The saved output in *3.EDA.ipynb* was created using an earlier version of *Nhanes_Cleaned.csv* that contained 28 columns. A later update to Notebook 2 removed three columns (SEQN, CYCLE, PHQ9_Score), leaving 25 columns in the current cleaned dataset. The number of respondents (13,731) and the values of the remaining features did not change. Because of this, the EDA findings and visualizations are still based on the same underlying data. Only the shape and column list shown near the beginning of the saved notebook reflect the earlier version.

5.1 Structural Checks

We first reviewed the overall structure and quality of the cleaned dataset. The notebook confirmed that there were no missing values and no duplicate rows, which was consistent with the final checks completed during data wrangling.

We also reviewed the column names and data types to identify which features were numeric or categorical before creating visualizations.

5.2 Target Variable Analysis

We reviewed the distribution of Depression_Flag and found that the target variable is imbalanced:

- 12,241 respondents (89.15%) were not flagged
- 1,490 respondents (10.85%) were flagged

The PHQ9_Score distribution was also right-skewed, with a skewness of approximately 1.79. About 29.72% of respondents had a PHQ-9 score of 0, and the median score was 2.

These results show that most respondents reported few or no depression symptoms, which helps explain the imbalance in our target variable.

5.3 Continuous Variable Distributions

We examined the distributions of the continuous features to identify skewness and other patterns in the data.

Age and the two sleep-hour variables were fairly symmetric. The income-to-poverty ratio variables showed some right skew, while sedentary time (PAD680) had a moderate right skew.

The largest skew was seen in alcohol-related variables. Average drinks per occasion (ALQ130) had a skewness of approximately 3.33, while binge-drinking days (ALQ170) had a skewness of approximately 6.43. This indicated that most respondents reported lower values, while a smaller number reported much higher alcohol use.

5.4 Categorical Variable Distributions of the "-1" Code

We also reviewed the distributions of the categorical variables. Gender was fairly balanced, with approximately 52.2% female and 47.8% male respondents. Non-Hispanic White was the largest race/ethnicity group at approximately 45.5%.

Some categorical features contained a value of -1. This value was intentionally created during data cleaning to represent respondents who did not provide an answer. We confirmed that these values were not the remaining missing values.

The percentage of -1 values varied across features, from less than 1% for variables such as DMBORN4 and SMQ020 to 28.18% for ALQ142. Because -1 represents its own response category, we kept it in the dataset instead of removing or imputing these values.

5.5 Relationships with the Target

We compared several demographic and socioeconomic features with Depression_Flag to see how they differed between respondents who screened positive and those who did not.

- Age: Respondents who screened positive were slightly younger on average (47.6 years) compared with respondents who did not (50.7 years)
- Income-to-poverty ratio: Respondents who screened positive had a lower average ratio (2.13) compared with those who did not (2.79)
- Gender: The positive screening rate was approximately 13.03% for females and 8.46% for males
- Education: The positive screening rate decreased from approximately 14.86% for respondents with less than a 9th-grade education to 6.11% for college graduates

While these differences were not extremely large, gender, education, and income-to-poverty ratio showed clearer relationships with the target than age.

5.6 Correlation Analysis

We used a correlation matrix to examine relationships between the continuous variables.

No individual continuous feature had a strong correlation with PHQ9_Score. The two income-to-poverty ratio variables had the strongest relationships with correlations of approximately -0.15 to -0.16.

- We also found some correlation between predictor variables:
  - INDFMPIR and INDFMPI had a strong correlation of approximately 0.82
  - The weekday and weekend sleep-hour variables had a moderate correlation of approximately 0.57

These results helped identify features that could contain similar or overlapping information before modeling.

5.7 Low-Variance Check

The low-variance analysis identified SMAQUEX2 as having only one unique value across the entire dataset, with a standard deviation of 0.0.

Since the feature does not vary between respondents, it does not provide useful information for predicting the target. No other predictor showed the same zero-variance issue.

5.8 How These Findings Carry Into Later Steps

The findings from EDA helped guide several decisions in the later stages of the project:

- The 89/11 class imbalance showed that accuracy alone would not be enough to evaluate our models. Because of this, we also used metrics such as PR-AUC, recall, precision, and F1
- The zero-variance feature SMAQUEX2 was removed during the feature engineering in Notebook 4
- Highly related or redundant features identified during our analysis were reviewed and removed where appropriate before model training
- The -1 category for unanswered categorical questions was kept as its own category during feature encoding rather than being treated as missing data

6. Feature Engineering and Data Splitting - 4.FeatureEngineering.ipynb

The feature engineering notebook prepares the cleaned dataset for machine learning. It removes features that are not useful for modeling, splits the data into training, validation, and test sets, and creates three different versions of the feature set based on the preprocessing needed for each model.

6.1 Input Path Behavior

Notebook 4 uses Nhanes_Cleaned.csv, which was created during the data wrangling stage.

One thing to note is that Notebook 4 currently looks for Nhanes_Cleaned.csv in the directory where the notebook is being run, while Notebook 2 saves the file inside Nhanes_RawData/. Because of this, the file path may need to be updated when reproducing the pipeline from the repository root.

The DataSplit/ folder contains the completed outputs from the feature engineering process used for our modeling.

6.2 Zero-Variance Column Removal

Before splitting the data, we checked the features for zero variance. As identified during EDA, SMAQUEX2 contained the same value for every respondent.

Since a feature with no variation cannot provide useful information to the models, SMAQUEX2 was removed before modeling.

6.3 Train/Validation/Test Split

The cleaned dataset is divided into training, validation, and test sets using scikit-learn's train_test_split.

The split is performed in two steps:

- First, 15% of the full data set is set aside as the test set
- The remaining 85% is then split again so that 15% of the original dataset becomes the validation set and approximately 70% remains for training

Both split operations use random_state = 4086 so the same split can be reproduced.

The final proportions are therefore approximately:

Split	**Percentage of Original Dataset**
Training	70%
Validation	15%
Test	15%

The stratify option is not used when creating the splits, so the code does not specifically control the class balance of Depression_Flag across the three datasets.

The resulting split contains 9,611 training rows, 2,060 validation rows, and 2,060 test rows, corresponding to approximately 70%/15%/15% of the full dataset.

6.4 Three Parallel Feature Encodings

Different machine learning models require different types of preprocessing. To account for this, we created three versions of the same training, validation, and test splits. This allows each model to use the preprocessing that works best for that type of algorithm while keeping the same respondents in each split.

Encoding	Preprocessing	Models
CatBoost	No one-hot encoding or scaling is applied because CatBoost can handle categorical features directly	CatBoost
One-hot encoded	Categorical features are converted using one-hot encoding and no scaling is applied	Random Forest, XGBoost
One-hot encoded & standardized	Categorical features are one-hot encoded, and the data is then standardized using StandardScaler	Logistic Regression, SVM

The one-hot encoder is fit using the training data only and then applied to the validation and test sets. The same process is used for StandardScaler. This prevents information from the validation or test sets from being used during preprocessing.

The target variable is also saved separately for each split and shared across all three encodings:

- Nhanes_Target_Train.csv
- Nhanes_Target_Val.csv
- Nhanes_Target_Test.csv

This keeps the train, validation, and test groups consistent across all five models.

6.5 Survey Weight Column (WTMEC_COMB) in the Split Files

The combined NHANES survey weight, WTMEC_COMB, is still included in the datasets created by Notebook 4. However, it is not used as a predictor when training any of the five models.

The survey weight represents how each respondent contributed to estimates of the larger U.S. population. Since it is a survey-design variable rather than one of the demographic, socioeconomic, or lifestyle features being used for prediction, it is removed from the feature set in each modeling notebook before training.

The column remains in the files stored in DataSplit/, but it is not included as an input feature in the trained models.

7. Machine Learning Models - 5.Modeling_*.ipynb

Five machine learning models were trained and evaluated for this project: Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, CatBoost.

The models use the different feature sets created during feature engineering based on their preprocessing needs. Random Forest and XGBoost use the one-hot encoded data, Logistic Regression and SVM use the one-hot encoded and standardized data, and CatBoost uses the version that keeps the categorical features in their original form.

Each modeling notebook follows the same general process of loading the appropriate data, removing features that are not used for prediction, training and tuning the model, selecting a classification threshold using the validation set, and evaluating the final model on the test set.

Because the target variable is imbalanced, we evaluate the models using multiple metrics instead of accuracy alone. These include PR-AUC, ROC-AUC, recall, precision, F1 Score, and accuracy.

7.1 Shared Preprocessing: Redundant Column Removal

Before training, every model notebook applies the same clean up to its own encoded copy of the data so all five models see equivalent information. The survey weight column (WTMEC_COMB) is removed, since it describes how NHANES sampled a person rather than anything about the person. A few duplicate information columns are also removed: the coarser race/ethnicity variable, both extra poverty index variants, and a duplicate drinking frequency measure. After this cleanup every model that uses the one-hot encoded data trains on the same 54 features.

7.2 Logistic Regression

Logistic Regression serves as the project's interpretable baseline. It trains on the standardized one-hot encoded split since it's a distance based linear model and needs features on comparable scales. Class imbalance is handled with class_weight="balanced". A decision threshold is chosen by maximizing F1 on the validation set then that same threshold is used for the final test set evaluation. On the test set, it reached an accuracy of 0.7820, precision of 0.2415, recall of 0.4255, and an F1-score of 0.3082.

7.3 Random Forest

Random Forest trains on the one-hot encoded unstandardized split since tree based models split on threshold and don't need scaled inputs. Like Logistic Regression it uses class_weight="balanced" to account for the class imbalance and a validation tuned F1 threshold carried through to test. Feature importance is measured by permutation importance rather than the model's built-in importance score since built-in importance is biased toward high cardinality one-hot groups. For example, the drinking frequency variable ALQ121 alone expands into 11 dummy columns which can dominate built-in importance without genuinely being predictive. On the test set, Random Forest reached the highest accuracy of the five

models (0.7920) but the lowest recall (0.377) and F1-score (0.2655) the clearest example in the whole project why accuracy alone is misleading on an imbalanced target.

7.4 Support Vector machine (SVM)

SVM trains on the standardized, one-hot encoded split (the same 54 features as Logistic Regression and Random Forest), using class_weight="balanced" to account for the class imbalance. Rather than a small hand-picked grid this code runs an extensive search across all four SVM kernel types linear, RBF, sigmoid, and polynomial tuning C (regularization strength) for every kernel, gamma for all but linear, and polynomial degree where relevant for 343 total parameter combinations evaluated using the validation set as a single held out fold rather than k-fold cross validation. The winning combination was an RBF kernel with c=10 and gamma=0.001. Raw SVM is not a calibrated probability, the winning model was wrapped in a CalibratedClassifierCV and refit so it produces genuine probability estimates the way the other models do. This turns out to matter a lot here: at the default 0.5 threshold, the calibrated model's validation recall was only 0.0213; it almost never predicted the positive class at that cutoff. After tuning the threshold on validation landing on 0.1706 which is far lower than the other models' thresholds, test set performance became strong with accuracy at 0.7864, precision at 0.2577, recall at 0.4638, and F1-score at 0.3313 which was the highest in all five models.

7.5 XGBoost

XGBoost also trains on the one-hot encoded, unstandardized split. Class imbalance is handled through XGBoost's scale_pos_weight parameter, which is set to the ratio of negative to positive training examples. A small grid over max_depth, learning_rate, and n_estimators was tuned on validation PR-AUC selecting max_depth=3, learning_rate=0.05, n_estimators=200 as the best combination. As with the other models, a validation-tuned F1 threshold (0.5808) carried through to the single test set evaluation. On the test set, XGBoost reached an accuracy of 0.7723, precision of 0.2365, recall of 0.4468, and an F1-score of 0.3093 the second highest recall of the five models. The notebook also includes a runtime check that raises an error if any ID, survey design, or PHQ-9 derived column is present in the feature set rather than only assuming the upstream cleaning steps were correct.

7.6 CatBoost

CatBoost is the only model that does not use one-hot encoding; it trains on a separate 18-column split that keeps 11 categorical variables (gender, race, education, marital status, alcohol use, smoking, etc.) in their native form as strings, letting CatBoost handle the categorical splitting internally rather than expanding each one into dummy columns. The same survey weight and duplicate columns are dropped as in every other model. Class imbalance is handled with auto_class_weights="Balanced". A baseline model (depth=6, learning_rate=0.05, iterations=500) was evaluated first (validation PR-AUC 0.2497), then a small grid over depth (4, 6, 8) and learning_rate (0.03, 0.05, 0.1) was tuned by validation PR-AUC the baseline settings turned out to already be the best combination in the grid. A validation-tuned F1 threshold of 0.5763, precision 0.2257, recall 0.4638, F1 0.3036, and PR-AUC 0.2723 the highest PR-AUC and tied for the highest recall of all five models, which is why it was ultimately selected as the project's final model.

7.7 Per-Model Scoreboard Files

Each model notebook writes its results to a shared Models/model_scoreboard.csv, using a common score_model() function which computes PR-AUC, ROC-AUC, recall, precision, F1, and accuracy. It also computed a Scoreboard class that appends each model's rows to the shared file. This is what lets all five models be compared side-by-side in one final table.

8. Model Evaluation and Comparison - 6.ModelComparison.ipynb

The final model comparison is completed in Notebook 6. This notebook loads all five trained models and evaluates them using the appropriate test datasets created during feature engineering. Using the same evaluation process for all five models allows for us to compare their performance and select a final model.

The final comparison results are also saved in the Models/ folder, including final_model_comparison.csv, course_required_metrics.csv, and final_model_card.md.

8.1 Project Plan Checks

Before the final model comparison, Notebook 6 checks how the completed project compares with the goals in our original project plan.

First, the notebook confirms that all five models are available: Logistic Regression, Random Forest, SVM, XGBoost, and CatBoost.

The notebook also compares the features used by the models with the predictors originally proposed in our Project Topic Form. Of the ten predictor areas originally planned, five are included in the final modeling data:

- Age → RIDAGEYR
- Sleep duration → SLD012, SLD013
- Physical activity → PAD680
- Education → DMDEDUC2
- Income → INDFMPIR

Five originally planned predictor areas are not included in the final modeling dataset: BMI, employment, general health, chronic diseases, and healthcare access.

These differences between the original plan and final implementation are documented in Models/project_goal_check.csv and are included as a limitation of the project.

8.2 Feature-Set Checks

Notebook 6 also checks the feature sets used by each trained model before comparing their performance.

The purpose or goal of this step was to make sure that variables related directly to the PHQ-9 target are not included as predictors and that each model uses the expected features set for its encoding.

The final models use:

- 54 features for Logistic Regression, Random Forest, SVM, and XGBoost
- 18 features for CatBoost

No target-leakage variables were identified in the feature sets used by the final models. The results of this check are saved in Models/feature_set_audit.csv.

8.3 Evaluation Metrics

Because Depression_Flag is imbalanced, we use several evaluation metrics rather than relying on accuracy alone.

Metric	**Meaning in Our Project**
Accuracy	Percentage of all respondents classified correctly
Precision	Of the respondents predicted to screen positive, the percentage correctly identified by the model
Recall	Of the respondents who actually screen positive, the percentage correctly identified by the model
F1 Score	Combines precision and recall into one score to measure the balance between the two
PR-AUC	Measure the model's precision-recall performance across different classification thresholds. This is used as a primary comparison metric because of the positive class presents a smaller portion of the dataset.
ROC-AUC	Measures how well the model separated the two classes across different thresholds

Accuracy is still reported, but it is not used by itself to determine the best model. Since approximately 89% of respondents are in the negative class, a model could achieve high accuracy by predicting the majority class while doing a poor job of identifying respondents who screen positive.

8.4 Final Model Selection

The models are primarily compared using test-set PR-AUC. If multiple models are within 0.01 PR-AUC of the highest-performing model, recall is used as the next comparison.

Recall is important for this project because the purpose of the model is screening. We want to consider how well the model identifies respondents who actually screen positive, rather than focusing only on overall accuracy.

Accuracy, precision, recall, F1, PR-AUC, and ROC-AUC are still reported for every model so that performance can be compared across multiple measures.

8.5 Final Test-Set Results

The final test-set results for all models are shown below:

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**PR-AUC**	**ROC-AUC**
CatBoost	0.7573	0.2257	**0.4638**	0.3036	**0.2723**	0.6761
XGBoost	0.7723	0.2365	0.4468	0.3093	0.2716	**0.6813**
SVM	0.7864	**0.2577**	**0.4638**	**0.3313**	0.2580	0.6795
Logistic Regression	0.7820	0.2415	0.4255	0.3082	0.2444	0.6781
Random Forest	**0.7932**	0.2232	0.3277	0.2655	0.2327	0.6736

The results show that no single model performs best across every metric. CatBoost has the highest PR-AUC (0.2723), while SVM has the highest F1 score (0.3313) and ties CatBoost for the highest recall (0.4638). Random Forest has the highest accuracy (0.7932), while XGBoost has the highest ROC-AUC (0.6813).

Ultimately, we decided to evaluate the models using multiple metrics rather than accuracy alone when comparing their performance.

8.6 Comparison Across Metrics

The model rankings change depending on which evaluation metric is used. CatBoost ranks first based on PR-AUC, while SVM performs best based on F1 score. CatBoost and SVM also have the same highest recall of 0.4638.

These differences show that the model performance depends on what type of error or performance measure is considered most important. For this project, PR-AUC remains the primary metric because of the class imbalance, with recall used as an additional consideration for the screening task.

8.7 Validation and Test Performance

We also compared validation and test PR-AUC to see how well each model's performance carried over to unseen test data.

The changes from validation to test PR-AUC were:

- CatBoost: -0.0226
- XGBoost: -0.0332
- SVM: 0.0169
- Logistic Regression: -0.0102
- Random Forest: 0.0110

The differences are relatively small across all five models. This suggests that the models performed reasonably consistently when moving from the validation data used during the tuning to the held-out test data.

8.8 Final Model Selection

Based on the model selection process, CatBoost was selected as the final model. This is because it achieved the highest test PR-AUC at 0.2723. XGBoost was close behind 0.2716, placing it within the 0.01 comparison range. CatBoost had the higher recall of the two models (0.4638 compared with 0.4468), which supported its selection as the final model.

The final CatBoost model is saved as: Models/final_model.joblib

The model selection and evaluation results are also documented: Models/final_model_card.md

At its tuned classification threshold of 0.5763, CatBoost produced the following confusion matrix on the test set:

	Predicted: No	Predicted: Yes
Actual: No	1,451	374
Actual: Yes	126	109

(CatBoost confusion matrix on the test set)

Out of the 235 respondents in the test set who screened positive, CatBoost correctly identified 109, resulting in a recall of approximately 46.4%. Of the respondents the model predicted as positive, approximately 22.6% actually screened positive.

8.9 Final Model Limitations

There are several limitations that should be considered when interpreting the final model:

- PHQ9_Score and the individual PHQ-9 questions are not used as predictors because they are used to create the target variable and including them would introduce target leakage
- Model thresholds were selected using validation data rather than the test set
- The model is based on self-reported NHANES survey information and is intended as a screening tool, not a medical diagnosis

- Five predictor areas included in the original project plan (BMI, employment, general health, chronic diseases, and healthcare access) are not included in the final modeling dataset
- The five models have relatively close PR-AUC results, meaning that the differences between the highest-performing models are small
- Precision remains relatively low across the models, meaning that some respondents flagged by the models will not actually screen as positive

In conclusion, the final results show that the models are able to identify some patterns related to depression screening outcomes, but there is still room for improvement. Adding additional relevant health and socioeconomic predictors could potentially improve future versions of the model.

9. Dependencies and Environment

9.1 Language and Notebook Environment

The project was developed in Python using Jupyter notebooks. Since the project was completed collaboratively, different notebooks were worked on and run by different group members on their own computers. Because of this, some saved notebook outputs may show different file path formats depending on the operating system and environment used. However, the project notebooks use relative paths for the main project files and folders where applicable.

9.2 Libraries Actually Imported

The project uses several Python libraries throughout the data preparation, modeling, evaluation stages.

Library	**Used For**
Pandas, numpy	Data loading, cleaning, transformation, and analysis
pyreadstat	Reading the NHANES .xpt SAS transport files
openpyxl, xlsxwriter	Reading and writing Excel files
matplotlib, seaborn	Creating EDA, model evaluation, feature importance, and comparison visualizations
scikit-learn	Data splitting and preprocessing with train_test_split, OneHotEncoder, and StandardScaler; training Logistic Regression, Random Forest, and SVM; hyperparameter tuning; SVM probability calibration; permutation importance; and model evaluation metrics

xgboost	Training the XGBoost classifier
catboost	Training the CatBoost classifier and handling categorical features
joblib	Saving and loading trained machine learning models: Logistic Regression, Random Forest, SVM, XGBoost, and CatBoost
glob, os, re, time, warnings, textwrap	File handling and other supporting Python utilities

9.3 Requirements for Running the Notebooks

The notebooks should be run in a pipeline order because later stages use files created by earlier notebooks. The main order is:

1. RawDataMapping.ipynb
2. DataWrangling.ipynb
3. EDA.ipynb
4. FeatureEngineering.ipynb
5. The four 5.Modeling_*.ipynb notebooks
6. ModelComparison.ipynb

The repository folder structure should also remain consistent because the notebooks use relative file paths to access data and saved outputs in folders such as Nhanes_RawData/, DataSplit/, Models/, and Figures/.

One file path difference should be noted when running the pipeline. Notebook 2 saves the cleaned dataset to: /Nhanes_RawData/Nhanes_Cleaned.csv. Notebook 4 currently looks for: ./Nhanes_Cleaned.csv. Because of this, the Notebook 4 input path may need to be updated or the CSV placed in the location expected by the notebook before running it.

The project also uses random states to make results reproducible. Notebook 4 uses random_state=4086 when creating the training, validation, and test sets. The modeling notebooks also use random_state=42 where applicable during model training and evaluations.

Finally, the folders needed for saved outputs should be available before running the later stages of the pipeline. Notebook 4 creates the data splits used for modeling and the modeling notebooks save trained models and evaluation results that are later used by Notebook 6 for the final model comparison.

10. Pipeline Summary

End-to-end, the project pipeline runs as follows:

Raw NHANES Data → Mapping → Cleaning → EDA → Feature Engineering → Train/Test Split → Model Training → Evaluation → Model Comparison

Stage	**Notebook**	**Key Output**
Raw NHANES Data	*(Nhanes_RawData/*/xpt)*	(Nhanes_RawData/*.xpt) 16 raw .xpt files across 8 topics × 2 cycles
Mapping	*1.RawDataMapping.ipynb*	Nhanes_Merged_Raw.csv (27,493 x 118), Nhanes_Mapped.xlsx
Cleaning	*2.DataWrangling.ipynb*	Nhanes_Cleaned.csv (13,731 x 25), Nhanes_Cleaned.xlsx
EDA	*3.EDA.ipynb*	Distribution, imbalance, correlation, and low-variance findings (Section 5)
Feature Engineering	*4.FeatureEngineering.ipynb*	12 CSVs in DataSplit/ (3 encodings x train/val/test, plus target vectors)
Train/Val/Test Split	*4.FeatureEngineering.ipynb*	9,611 / 2,060 / 2,060 rows (about 70/15/15, unstratified)
Model Training	*5.Modeling_rf_logreg.ipynb, 5.Modeling_svm.ipynb, 5.Modeling_XGBoost.ipynb, 5.Modeling_CatBoost.ipynb*	5 fitted models in Models/*.joblib, permutation-importance CSVs, Figures/*.png
Evaluation	*6.ModelComparison.ipynb*	Independent re-scoring of all 5 models; Models/final_model_comparison.csv, Models/course_required_metrics.csv
Model Comparison / Selection	*6.ModelComparison.ipynb*	Models/final_model_card.md, Models/final_model.joblib - CatBoost selected (Section 8.8)